Project 3 (C++): Given a connected undirected graph, G=<N, E>, using the Prim's algorithm to construct the Minimum Spanning Tree (MST) of G. (Not a hard project, you are given 3 and ½ day to implement it.)

Prim's algorithm uses two sets, setA and setB to construct an MST of G. Initially, setA contains only one node (can be any node in G and setB contains the rest of nodes.

You will be given 2 data files: data1 and data2; data1 is a small graph, and data2 is a larger graph.
What you need to do as follows:

1) Use a blank paper, draw the graph for data1 on the top of the page. Then draw the sequence of boxes for SetA and SetB with 3 as the initial node in SetA, using Prim's algorithm to find the MST as illustrated in class.
2) Implement your program based on the specs given below.
3) Run and debug your program with data1 (using 3 as the initial node in setA and the rest in setB) until your program produces the same MST as in your illustration.
4) Run your program 6 times with data2 using 1, 3, 5, 7. 9, 11 as the initial node in setA.

**Include in your hard copy:
- cover page with algorithm steps.
- hand drawn illustration of 1) in the above. -2 if omitted.
- source code.
- outFile1 and MSTFile with data1 using 3 as initial SetA.
- outFile1 and MSTFile with data2 using 1, 3, 5, 7, 9, and 11 the initial node in setA.

****

Language: C++
****

Project name: Prim's Minimum Spending Tree algorithm
Project points: 10pts
Due Date: Soft copy and hard copies:
10/10 (on time): 3/1/2026 Monday before midnight (11:59pm).
-10/10 (non-submission): 3/1/2026 Monday after midnight.

*** All submission MUST include Soft copy (*.zip) and hard copy (*.docx) in the same email attachments with correct email subject as stated in the project submission requirement; otherwise, your submission will be rejected.

Email subject: (323) first name last name <Project 3: Prim's Minimum Spending Tree algorithm (C++)>

*** Inside the email body includes:
- Your answer to the five questions (-2 if omitted)
- Screen recoding link. (-2 without the recording!)

*** Place your screen recording link in your project submission email body below the 5 questions.

****

I. Inputs: There are two inputs to the program
a) inFile (argv [1]): A text file contains a connected undirected graph, as a list of edges with costs, <n_i, n_j, cost>.
The first text line in the file is the number of nodes in G, follows by a list of triplets, <n_i, n_j, cost>
For example:

```
5          // there are 5 nodes in the graph
1  5 10  // an undirected edge: i.e., an edge <1, 5, 10> and an edge <5, 1, 10>
2  3  5   // an undirected edge: i.e.,  an edge <2, 3, 5> and an edge <3, 2, 5>
1  2  20
3  5  2
:
:
```

b) nodeInSetA (argv [2]) // user select a node to be in setA.

****

II. Output:

a) MSTfile (argv [3]): The result of Prim's MST in text file in the following format:

```
==============================
** There are 5 nodes in the input graph
** Initial node in SetA =
** Edges in MST are given below
        2  3  5 //  an edge <2, 3, 5>
        3  5  2
        :
 ** The total cost =
==============================
```

b) outFile1 (argv [4]): as program dictates

****

III. Data structure:

****

- An uEdge class
  - (int) Ni // node ID.
  - (int) Nj // node ID.
  - (int) cost // the cost of the edge <Ni, Nj>.
  - (uEdge *) next

  Methods:
  - constructor (…) // to create a new uEdge node with (Ni, Nj, cost, null)
  - printEdge (edge, outFile1) // print the given edge info to outFile1 in the following format.
    <edge's Ni, edge's Nj, edge's cost, edge's next>

- A PrimMST class
  - (int) numNodes //number of nodes in G.
  - (int) nodeInSetA // get from argv [2]
  - (char *) whichSet // a 1-D char array, size of numNodes +1, dynamically allocated.
    // to indicate which set each node belongs to ('A' for setA or 'B' for setB)
  - (uEdge *) edgeListHead // The list head of a sorted linked list of uEdge of the input graph edges
    //in ascending order w.r.t. cost; so that to find a min cost edge would be simple,
    // i.e., at the front of the list, as long as the two nodes in the edge are not in the same set.
    // Initially, it points to a dummy node <0, 0, 0, null> when created.
  - (uEdge *) MSTlistHead // The list head of an un-sorted linked list of uEdge nodes in Prim's MST.
    // Initially, it points to a dummy node <0, 0, 0, null>
  - (int) totalMSTCost // initially set to zero

  Methods:
  - (uEdge) findSpot (edgeListHead, newEdge) // find the spot where newEdge to be inserted after Spot. Search key
    // is cost, in ascending order; similar to findspot (…) in your project 2.
  - insertOneNode (Spot, newEdge) // insert newEdge between Spot and Spot.next.
    // newEdge-> next ← Spot->next
    // Spot-> next ← newEdge.
  - (uEdge) removeEdge (…) // The method searches edge nodes in edgeListHead list, if list is not empty,
    // finds the first edge, e: <Ni, Nj, cost> where Ni and Nj are NOT in the same set
    // i.e., whichSet[e.Ni] != whichSet[e.Nj] **and** one of Ni or Nj is in SetA, i.e.,
    // either whichSet[e->Ni] == 'A' or whichSet[e->Nj] == 'A'.
    // Then, remove edge node <Ni, Nj, cost> from the edge list and returns e.
    // **On your own.**
  - updateMST (…) // **See algorithm below**.
  - pushEdgeToMST (…) // push edge on the top of MSTlistHead after dummy. **//stack operation. On your own.**

- printSet (…) // print the whichSet array to **outFile1**, in two text-lines with index on the top
  // and whichSet[i] on the second text-line; for example

```
1  2  3  4  5
A  B  A  B  A
```

- printEdgeList (…) // print edges in edgeListHead to **outFile1** with the following format:

  // edgelistHead → <0, 0, 0, Ni_1> → <Ni_1, Nj_1, edgeCost, Ni_2> → < Ni_2, Nj_2, edgeCost, Ni_3> …

- printMSTList (…) // print nodes in the list point by MSTlistHead to **outFile1** with the following
  // MSTlistHead → <0, 0, 0, Ni_1> → <Ni_1, Nj_1, edgeCost, Ni_2> → < Ni_2, Nj_2, edgeCost, Ni_3> …
- printMST (…) // print the final MST to MSTfile in the format given in the above, under section II, MSTfile.
- (bool) isDone (…) // returns true if whichSet are all 'A', otherwise returns false.
- updateMST (newEdge) // **see algorithm below**.

****

IV. main (…)

****

Step 0: inFile, outFile1, MSTfile ← open via argv[]
numNodes ← get from inFile
nodeInSetA ← get from argv [2]
whichSet [] ← dynamically allocated, size of numNodes+1, and initialize all to set 'B'
whichSet [0] ← 'A' // although we do not use index 0.
whichSet[nodeInSetA] ← 'A' // now, setA has only one node, nodeInSetA.
printSet (whichSet, outFile1) // with caption.
edgelistHead ← get an uEdge as the dummy node <0, 0, 0, null> for it to point to.
MSTlistHead ← get an uEdge as the dummy node <0, 0, 0, null> for it to point to.
totalMSTCost ← 0

// Step 1 to Step 3 are constructing the linked list of edges in ascending order with respect to edge costs.

Step 1: Ni ← read from inFile.
Nj ← read from inFile.
edgeCost ← read from inFile.
newEdge ← create a new uEdge node with (Ni, Nj, edgeCost, null)
Spot ←findSpot (edgelistHead, newEdge)
insertOneNode (Spot, newEdge)

Step 2: outFile1 ← "In main () print the list of edges"
printEdgeList (edgelistHead, outFile1)

Step 3: repeat step 1 to step 2 until the inFile is empty.

// Step 4 to Step 9 are constructing MST, need not be sorted.

Step 4: e ← removeEdge (edgelistHead)
Step 5: printEdge (e, outFile1) // with caption: "**The removed edge is:" // print edge e.
Step 6: updateMST (MSTlistHead, e)
Step 7: printSet (whichSet, outFile1) // with caption: "** Below shows nodes in whichSet **"
Step 8: printEdgeList (edgeListHead, outFile1) // with caption: "** Below is the sorted edge list **"
printMSTList (MSTlistHead, outFile1) // with caption: "Below is the intermediate constructed MST list **"
Step 9: repeat step 4 – step 8 until isDone (whichSet)
Step 10: printMST (MSTlistHead, MSTfile)
Step 11: close all files.

****

V. updateMST (MSTlistHead, edge)

****

Step 1: pushEdgeToMST (MSTlistHead, edge)
Step 2: totalMSTCost += edge.cost
Step 3: if whichSet [edge.Ni] == 'A' // Ni is in setA,
whichSet [edge.Nj] ← 'A' // re-assign Nj from setB to setA
else
whichSet [edge.Ni] ← 'A' // re-assign Ni from setB to setA